# Najčešće postavljana pitanja i odgovori

1. **Pitanje:** Šta je tema projektnog zadatka?
   **Odgovor:** Veb aplikacija za deljenje *Docker* slika. Najbolja asocijacija za ono šta bi trebalo napraviti jeste jednostavnija verzija *DockerHub* platforme.
2. **Pitanje:** Da li je projekat timski? Ukoliko jeste, koliko studenata čini tim?
   **Odgovor:** Da! Tim čini 3-5 studenata, preporuka je da to bude 4.
3. **Pitanje:** Koje funkcionalnosti treba da podrži aplikacija?
   **Odgovor:** Detaljniji opis funkcionalnosti i logički vezanih nefunkcionalnih zahteva:
   - Priprema sistema za korišćenje (*Setup*).
   - Registrovanje, autentifikacija i autorizacija korisnika - Potrebno je da sistem podrži bar sledeće tri uloge: administrator (jedan superadminsitrator i više običnih), običan autentifikovani korisnik, neautentifikovani korisnik. Super administrator se kreira u sklopu *Setup* procedure i dodaje druge administratore. Običan autentifikovani korisnik se samostalno registruje u sistem. Svaki registrovani korisnik može da vidi i izmeni profil, a običan korisnik može i da vidi spisak repozitorijuma za koje je označio da su bitni (*star*).
   - Napredna pretraga javnih repozitorijuma u sistemu po uzoru na *DockerHub* pretragu (*Explore* sekcija).
   - Rad sa repozitorijumima od strane običnog autentifikovanog korisnika (*Repositories* sekcija).
   - Rad sa organizacijama od strane običnog autentifikovanog korisnika (*Organizations* sekcija).
   - Rad sa zvaničnim repozitorijumima od strane administratora.
   - Rad sa običnim korisnicima od strane administratora.
   - Praćenje rada sistema od strane administratora (*Analytics* sekcija).
   - Bonus zahtev: Označavanje značajnih repozitorijuma (*star*).

   Dodatna objašnjenja možete pronaći u odgovorima na pitanja koja slede.
4. **Pitanje:** Koji su sve funkcionalni i nefunkcionalni zahtevi vezani za pripremu sistema za korišćenje (*Setup*)?
   **Odgovor:** Kada se pokrene sistem prvi put, generiše se jedan super administrator nalog. Predefinisana (*default*) šifra super administratora se

generiše i upisuje u datoteku. Putanju do ove datoteke navesti u uputstvu za pokretanje aplikacije. Sa ovom šifrom se super administrator prijavljuje i prilikom prve prijave mora da promeni lozinku. Ako to ne uradi, ne dobija pristup sistemu. Nakon promene lozinke, dobija pristup svim funkcijama sistema. Može da registruje druge administratore u sistemu (samo superadministrator ima ovu mogućnost).

5. **Pitanje:** Koji su sve funkcionalni i nefunkcionalni zahtevi vezani za *Explore* sekciju?

   **Odgovor:** Potrebno je ispuniti sledeće zahteve:
   - Svi korisnici mogu da pretraže sve javne slike u sistemu, a rezultati pretrage se sortiraju po relevantnosti (samostalno odrediti metriku relevantnosti)
   - Moguce je filtrirati rezultate po značkama (*Docker Official Image, Verified Publisher, Sponsored OSS*; više o značenju i dodeli znački u nastavku).
   - Izborom repozitorijuma moguće je videti sve njegove detalje (opis, tagovi, koliko zvezda ima itd.).
6. **Pitanje:** Šta podrazumeva rad sa repozitorijumima od strane običnog autentifikovanog korisnika (*Repositories* sekcija)?

   **Odgovor:** Podrazumeva sledeći skup funkcionalnosti za upravljanje repozitorijumima:
   - Početna strana sekcije - Korisnik može da vidi spisak repozitorijuma koji su njegovi ili pripadaju organizacijama čiji je korisnik član, može da filtrira ovaj spisak repozitorijuma, izborom nekog repozitorijuma može da vidi detalje.
   - Kreiranje novog repozitorijuma - Repozitorijum ima naziv, kratak opis i može biti privatan ili javan (vidljivost), a korisnik bira da li je repozitorijum lični ili pripada organizaciji kojoj taj korisnik pripada.
   - Izmena podešavanja repozitorijuma - Korisnik može da promeni opis ili vidljivost
   - Upravljanje tagovima - korisnik može da vidi sve tagove, da ih sortira i filtrira i da ih obriše. Dodavanje tagova se radi putem *docker push* komande iz terminala.
   - Brisanje repozitorijuma - Briše se repozitorijum i sve informacije koje mu pripadaju.

7. **Pitanje:** Šta podrazumeva rad sa organizacijama od strane običnog autentifikovanog korisnika (*Organizations* sekcija)?
   **Odgovor:** Podrazumeva sledeći skup funkcionalnosti za upravljanje organizacijama:
   - Početna strana sekcije - Korisnik može da vidi spisak organizacija kojima pripada, izborom neke organizacije može da vidi detalje.
   - Kreiranje nove organizacije - Organizacija ima naziv, kratak opis i sliku.
   - Izmena podešavanja organizacije
     - Vlasnik organizacije može da menja kratak opis i sliku.
     - Vlasnik organizacije može da deaktivira organizaciju čime se uklanjaju članovi, brišu svi repozitorijumi i informacije o organizaciji.
   - Članovi organizacije (*Members* podsekcija)
     - Svaki član organizacije može videti sve članove organizacije.
     - Vlasnik organizacije može da pretraži korisnike u aplikaciji i doda ih u organizaciju.
   - Timovi (*Teams* podsekcija)
     - Svaki član organizacije može videti sve timove.
     - Vlasnik organizacije može da pravi timove.
     - Vlasnik organizacije može da dodaje članove organizacije u timove, a za svaki tim se mogu definisati permisije *read-only*, *read+write* ili *admin* za neki repozitorijum koji pripada organizaciji:
       - *read-only* omogućava gledanje informacije repozitorijuma;
       - *read+write* omogućava *push* nove verzije brisanje tagova;
       - *admin* omogućava da se u potpunosti upravlja repozitorijumom kao da su članovi tog tima vlasnici.
     - Vlasnik organizacije može da izmeni naziv i opis tima.
   - Repozitorijumi organizacije (*Repositories* podsekcija)
     - Moguce je videti i pretražiti sve repozitorijume te organizacije, a u skladu sa pripadnošću timovima i njihovim permisijama

neki članovi organizacije mogu da upravljaju repozitorijumima (detalji objašnjeni u prethodnom pitanju).
    - Bilo koji član organizacije može da kreira novi repozitorijum koji pripada toj organizaciji.
8. **Pitanje:** Šta spada u rad sa zvaničnim repozitorijumima od strane administratora?
   **Odgovor:** Svi administratori mogu da kreiraju zvaničan repozitorijum i da upravljaju njima (objašnjeno u ranijim pitanja šta obuhvata upravljanje repozitorijumom). Zvaničan repozitorijum se razlikuje od običnih po tome što obični imaju naziv *prefix/naziv_repozitorijuma* (pri čemu je vrednost prefix-a korisničko ime ili naziv organizacije koja je kreirala repozitorijum), dok zvanični nema prefix već samo naziv i zvanični repozitorijum ima značku *Docker Official Image*.
9. **Pitanje:** Šta spada u rad sa običnim korisnicima od strane administratora?
   **Odgovor:** Svi administratori mogu da pretraže obične korisnike u sistemu. Administrator može običnom korisniku dodeliti značku *Verified Publisher* ili *Sponsored OSS*, koje se vide prilikom pregleda svih repozitorijuma (*Explore* sekcija) i u njegovim detaljima.
10. **Pitanje:** Šta spada u praćenje rada sistema od strane administratora (*Analytics* sekcija)?
    **Odgovor:** Svi administratori mogu da prate stanje sistema kroz analizu događaja. Aplikacija koju razvijete tokom svog rada generiše veliki broj logova. Logove bi trebalo čuvati u log datotekama. Sistem treba da periodično preuzima nove logove i šalje ih *elasticsearch* platformi na obradu. Pomoću *elasticsearch* platforme, omogućiti parsiranje i pretraživanje logova. Logove je moguće pretražiti po datumu i vremenu kada se dese (pre i posle nekog datuma i vremena), nivou (info, warning, error itd.) i tekstualnom sadržaju (sadržati reč ili frazu). Administrator putem korisničkog interfejsa može da definiše logički upit za pretragu logova koji ima više kriterijuma pretrage (koristeći operatore *AND*, *OR* i *NOT* i zagrade) i šalje ga aplikaciji koja ga pretvara u format upita koji podržava *elasticsearch* platforma, koja zatim vraća rezultate sortirane po relevantnosti koji se na kraju ovog procesa prikazuju administratoru na korisničkom interfejsu. Primer upita bi bio sledeći: *(log ima nivo warning OR log ima nivo error) AND teksutalni sadržaj sadrži frazu "error occured"*.

11. **Pitanje:** Šta predstavlja Bonus zahtev: Označavanje značajnih repozitorijuma (*star*)?
    **Odgovor:** Bonus zahtev predstavlja dodatni zahtev koji je moguće braniti isključivo ako se brani projekat u prvom roku i nosi 4 bodova. U slučaju da tim izgubi određeni broj bodova (odnosno ne ostvari maksimalnih 60), izradom ove funkcionalnosti može da nadoknadi deo izgubljenih bodova maksimalno 4). U drugom i trećem roku za odbranu projekta nije moguće ostvariti bodove za ovaj zahtev.
    Kada običan korisnik pronađe neki javni repozitorijum koji nije njegov lični i ne pripada organizaciji čiji je član, može da ga označi bitnim (*star*). Na stranici javnih repozitorijuma moguće je videti koliko različitih korisnika je označilo taj repozitorijum bitnim (broj *star*-ova). U sklopu korisničkog profila, običan autentifikovani korisnik može da vidi spisak svih repozitorijuma koje je označio da su mu bitni.
12. **Pitanje:** Koliko bodova nosi projektni zadatak i kakva je raspodela istih?
    **Odgovor:** Pogledati profesorovu prezentaciju i posebnu pažnju obratiti na slajd koji sadrži tabelu sa rasporedom bodova. Narednih nekoliko pitanja objašnjavaju detaljnije svaku od stavki.
13. **Pitanje:** Šta spada u stavku *model*?
    **Odgovor:** UML dijagram klasa koje čine model vaše aplikacije. Kao polaznu osnovu možete koristiti dijagram koji sa profesorom napravite u jednom od termina predavanja.
14. **Pitanje:** Šta sve podrazumeva stavka *implementacija projekta*?
15. **Odgovor:** Implementacija svih funkcionalnih i nefunkcionalnih zahteva.
16. **Pitanje:** Šta potpada pod stavku *implementacija testova*?
    **Odgovor:** Implementacija jediničnih (engl. *unit tests*) i integracionih testova (engl. *integration tests*). Poželjno je implementirati *end-to-end* testove, ali nije neophodno.
17. **Pitanje:** Šta obuhvata stavka *git*?
    **Odgovor:** Upotreba git alata za verzioniranje koda tokom razvoja aplikacije.
18. **Pitanje:** Na koji je način potrebno voditi git repozitorijum?
    **Odgovor:** Git repozitorijum voditi po preporučenom GitFlow modelu. Potrebno je da da imate *develop* granu na kojoj se nalazi kod koji se intenzivno razvija i koji često testiraju svi članovi tima. Kod na ovoj grani

**mora da radi**, a čine ga do datog trenutka implementirane funkcionalnosti koje su dovoljno dobro istestirane.

Razvoj nove funkcionalnosti radi se na posebnoj *feature* grani. Svaka *feature* grana odgovara tačno jednom *GitHub issue*-u. Ove grane predstavljaju alternativne tokove razvoja. Jednu funkcionalnost radi jedan student. Kada se funkcionalnost završi, potrebno je ovu granu spojiti sa develop granom putem pull *request*-a. *Feature* grane imenovati u formatu *feature-<naziv_grane>*[1].

U slučaju da primetite neki *bug*, tada pronalazite *issue*[2] koji odgovara funkcionalnosti, ponovo ga otvarate i kreirate odgovarajuću *bugfix* granu (format imenovanja je *bugfix-<naziv_grane>*[3]). Na njoj popravljate *bug* i radite inicijalno testiranje. Kada ste utvrdili da je problem otklonjen, kod spajate sa *develop* granom putem *pull request*-a.

Periodično, kod sa *develop* grane prebacujete na master granu. Master grana bi trebalo da sadrži samo *commit*-e koji se odnose na *release*-ove, tj. zvanične verzije Vaše aplikacije. Potrebno je imati minimalno onoliko verzija koliko i *milestone*-ova.

19. **Pitanje:** Šta obuhvata stavka *GitHub*?

    **Odgovor:** Upotreba *GitHub* platforme tokom razvoja aplikacije za menadžerske poslove, tj. podjednako raspoređivanje nadležnosti na članove tima. Drugačije rečeno, potrebno je da redovno vodite računa o *milestone*-ovima i *issue*-ima.

20. **Pitanje:** Koliko *GitHub milestone*-ova je neophodno napraviti i do kada su rokovi za izradu svakog?

    **Odgovor:** *Milestone* oslikava jedan bitan momenat životnog ciklusa vaše aplikacije. ~~Što se tiče ovog projekta možete da imate **2** milestone-a. Rokovi su: kontrolna tačka i predaja projekta (datumi i vreme biće naknadno specificirani u dogovoru sa profesorom).~~ Neophodno je da imate minimalno 1 milestone. *Milestone* sadrži *task*-ove (*GitHub issues*), koji su implementirani tokom tog *milestone*-a.

21. **Pitanje:** Na koji način pravimo *GitHub issue*-e (*task*-ove)?

---

[1] Ignorisati < i > karaktere.
[2] Način upotrebe issue-a objašnjen kasnije.
[3] Ignorisati < i > karaktere.

**Odgovor:** Jedan *task* treba da se odnosi na jednu funkcionalnost aplikacije (engl. *feature*), koja će biti razvijena na posebnoj *feature* grani. *Task*-ove je potrebno napraviti unapred za *milestone* koji je u toku. Zatim, svaki student uzima task po task (dodeljuje mu se *issue*), te započinje implementaciju odgovarajuće funkcionalnosti. Kada se implementacija funkcionalnosti završi i uspešno odradi inicijalno testiranje, *task* se zatvara, a kod prebacuje na *develop* granu. Nakon toga, ostale kolege treba da urade testiranje svih funkcionalnosti. Ukoliko se ustanovi problem u nekoj od njih, potrebno je ponovo otvoriti odgovarajući task (**ne praviti nov**).

22. **Pitanje:** Šta podrazumeva stavka SCM?
    **Odgovor:** Ukratko - sve ono što se tiče upravljanja konfiguracijom softvera aplikacije koju razvijate. Detaljnije - Od samog početka razvoja aplikacije, potrebno je da koristite neki od alata za *CI/CD* (engl. *Continuous Integration and Continuous Delivery*) koji ćete povezati sa *GitHub* platformom. Kao krajnji proizvod, korisniku dostavljate aplikaciju koju ste prethodno kontejnerizovali. Vaša aplikacija, koristiće i neke druge servise koji su takođe zapakovani u kontejnere, te je neophodno da dostavite konfiguraciju nekog od alata za orkestraciju, kako bi čitav sistem funkcionisao (detaljniji opisi slede).
23. **Pitanje:** Koje sve *build* planove treba specificirati u okviru *CI/CD* alata?
    **Odgovor:** Neophodno je podržati bar dva *build* plan, koji će se aktivirati prilikom sledećih događaja:
    - kreiranje novog *pull request*-a na *develop* grani: *Build plan* će pokrenuti sve testova Vaše aplikacije (*unit* i *integration*) i pokazati da li je *pull request* koji predstavlja novi *feature,* odnosno *bugfix* u redu integrisati sa *develop* granom.
    - kreiranje nove verzije aplikacije na *master* grani: *Build plan* će pokrenuti automatizovane testove. Ukoliko se svi testovi uspešno završe, aplikacija se pakuje u kontejner koji se šalje na neki od javnih registara kontejnera.

    Možete podržati i druge *build* planove, koje smatrate korisnim. Odluku o broju *build* planova donesite u zavisnosti od toga šta dobijate besplatno od *CI/CD* alata za koji se odlučite.

24. **Pitanje:** S obzirom na to da treba da razvijemo platformu sličnu *DockerHub*-u, na koji način treba da upravljamo repozitorijumima unutar aplikacije, tj. da li treba repozitorijume negde da *host*-ujemo?
    **Odgovor:** Postoje tri predefinisana načina i svaki od njih unosi određeni limit broja bodova koji propisuje stavka *Implementacija projekta*[4]:
    - Repozitorijume ne *host*-ujete nigde. Sve informacije unosite kroz odgovarajuće forme. Npr. simulirate *push* na Vašu platformu tako što ćete informacije o *image*-ima (naziv *image*-a, autora, tag, itd.) uneti kroz neku formu koju aktivirate na dugme nov *push*. Ovakav način donosi maksimalno 18 bodova.
    - Repozitorijume *host*-ujete na *DockerHub* platformi, a Vašu aplikaciju povezujete sa istom preko *DockerHub API*-a. Korisnik inicira zahtev za push operacijom, koji odlazi direktno na *DockerHub*, a zatim odgovor *DockerHub* platforme beležite u sistemu Vaše aplikacije. Ukoliko se odlučite za ovaj način, možete osvojiti najviše 21 bodova.
    - Repozitorijume *host*-ujete na nekom od gotovih container registry-a (npr. Harbor, Quay, JFrog Container Registry, Distribution) koje održavate sami. Znači neophodno je podesiti neki od ovih aplikacija da trči u nezavisnom kontejneru. Tada *push* operacija gađa aplikaciju u tom kontejneru. Vaša aplikacija kupi informacije iz te aplikacije putem API-a i prikazuje korisniku. Ukoliko se odlučite za ovaj način, možete osvojiti najviše 26 bodova.

      **Napomena:** Ukoliko se odlučite za primenu nekog od poslednja dva načina, morate ih samostalno sprovesti u delo. U pitanju su napredne funkcionalnosti namenjene studentima koji žele da iskažu posebno zalaganje na predmetu i tako konkurišu za najvišu ocenu. Ovo iziskuje visok stepen samostalnosti pri rešavanju problema, te se ne može računati na pomoć predmetnog asistenta prilikom suočavanja sa tehničkim problemima.
25. **Pitanje:** Koliko je minimalno kontejnera neophodno orkestrirati kako bi se u potpunosti zadovoljila stavka SCM?
    **Odgovor:** Minimalno 4. U jednom kontejneru izvršavaće se Vaša aplikacija. Ona mora komunicirati sa bazom, koja se izvršava u novom kontejneru.

[4] Uveden je primer push operacije, kao jedan od najčešćih scenarija upotrebe Vaše aplikacije.

Aplikacija bi trebalo da kešira određene rezultate u posebno prilagođenom *in-memory* skladištu u cilju poboljšanja performansi i povećanja propusnog opsega. Ovakvo skladište se izvršava u pripadajućem kontejneru. Takođe, ispred Vaše aplikacije treba da stoji *reverse prox*, koji se izvršava u posebnom kontejneru. Ukoliko studenti prepoznaju još neke elemente koji predstavljaju prikladna proširenja sistemu, mogu ih uvesti u isti i za njih kreirati kontejnere.

26. **Pitanje:** Da li postoji bodovna tabela koja preciznije ukazuje na to koliko svaka od prethodno pomenutih stavki nosi bodova?
    **Odgovor:** Pogledati sledeću tabelu[5].
27. **Pitanje:** Da li postoji minimalan broj bodova koji se mora osvojiti da bi se projekat smatrao uspešno odbranjenim?
    **Odgovor:** Potrebno je osvojiti minimalno 25 bodova, uz uslov da se mora osvojiti bar 6 bodova koji pripadaju funkcionalnostima označenim sa ***Osnovno*** u tabeli.
28. **Pitanje:** Šta je od tehnologija obavezno da se koristi?
    **Odgovor:** Nema obaveznih tehnologija. Studentima se ostavlja na volju da odaberu tehnologije shodno ličnim preferencijama. U nastavku sledi spisak preporučenih tehnologija:
    - Operativni sistema: *Linux*,
    - Radni okvir za razvoj veb aplikacija: *Django*,
    - Alat za testiranje: Alat za automatizovano testiranje Django radnog okvira,
    - *CI/CD* alat: *GitHub Actions*,
    - Alat za kontejnerizaciju: *Docker*,
    - Alat za orkestraciju: *Docker Compose*,
    - Javni registar kontejnera: *DockerHub*,
    - *Reverse proxy* server: *NGINX*,
    - *In-memory cache*: *Redis*.
    - Alati za modelovanje: draw.io, PlantUML

      **Napomena:** Ukoliko se studenti odluče za korišćenje nekih drugih tehnologija, tada ne mogu očekivati pomoć od asistenta ili profesora,

[5] Tabela je podložna izmenama.

prilikom rešavanja tehničkih problema uzrokovanih odabranom tehnologijom.

29. **Pitanje:** Kako izgleda izdavanje nove verzije projekta (release)?
    **Odgovor**: Tagovati *commit* na master grani.
30. **Pitanje:** Da li bismo mogli odmah da dokerizujemo aplikaciju ili je bolje da taj korak ostavimo za posle.
    **Odgovor:** Da. Nemam ništa protiv. Odluku prepuštam Vama.
31. **Pitanje:** Da li mi odeđujemo *milestone*-ove, odnosno da li su uopšte potrebni i koliko treba da ih imamo do odbrane projekta
    **Odgovor:** Pogledati odgovor na pitanje 19.
32. **Pitanje:** Da li imamo kontrolnu tačku i kada?
    **Odgovor:** ~~Da. Termin iste biće naknadno objavljen u dogovoru sa profesorom.~~ Ne. Zbog trenutne situacije nije moguće realizovati kontrolnu tačku, već će biti održana isključivo završna odbrana (u 3 roka).
33. **Pitanje:** Da li bih mogla u Dockerfile-u da dodam migraciju i da kreiram superusera, i ako bih mogla, kako da uradim tako nesto, posto sam videla kada samo dokerizujemo aplikaciju, to je okej, ali kada je pokrecemo preko docker-compose orkestratora, onda dolazi do problema jer aplikacija jos uvek nije povezana na bazu.
    **Odgovor:** Redosled startovanje kontejnera od strane Docker Compose-a nije zagarantovan. Stoga, kontejner sa Django aplikacijom mora sačekati da se startuje kontejner sa bazom. U sledećem primeru možete videti kako je moguće rešiti ovaj tip problema.
34. **Pitanje:** Šta znači inicijalno testiranje (iz odgovora na pitanje broj 17.), da li se to odnosi na pisanje testova ili testiranje koje podrazumeva da smo manualno proverili funkcionalnost?
    **Odgovor:** Podrazumeva oba.

~~35. **Pitanje:** Kako izgledaju kontrolne tačke?~~
    ~~**Odgovor:** Svi timovi treba ukratko da prezentuju (10-15 minuta) šta su do datog trenutka odradili. Prilikom prezentacije, osvrnuti se na sledeće:~~
    - ~~Šta je odrađeno?~~
    - ~~Zanimljivi problemi sa kojima ste se susreli?~~
    - ~~Kakav je plan za dalje?~~
    - ~~Nedoumice za nastavak izrade?~~

~~36. **Pitanje:** Da li se kontrolne tačke ocenjuju?~~

~~**Odgovor:** Ne, ali mogu doprineti sticanju boljeg utiska o zalaganju tima, što svakako može značiti tokom odbrane projekta.~~

37. **Pitanje:** Da li je bitan procenat završenosti milestone-a?
    **Odgovor:** Da! Težite da do krajnjeg roka milestone procenat završenosti tog milestone bude 100%. U tom slučaju se milestone smatra uspešnim. U suprotnom, došlo je do probijanja roka.
38. **Pitanje:** Da li moramo da razvijamo i docker client?
    **Odgovor:** Ne. Koristiti *docker client* (*docker* komandu terminala) kao i do sada.
39. **Pitanje:** Da li je potrebno da otvaramo issue za inicijalnu postavku projekta kao što su frontend, backend, testing framework, generički repozitorijum (mislim na sloj za pristup bazi podataka a ne domenski repozitorijum)? Ukoliko je potrebno, da li je u redu da napravimo neki 00 milestone koji će biti najkraći i u kojem ćemo samo postaviti projekat?
    **Odgovor:** Trebalo bi napraviti issue koji bi se ticao inicijalne konfiguracije, jer se i taj deo loguje. Ne morate praviti 00 milestone, već ga pridružiti prvom kreiranom.
40. **Pitanje:** Da li je potrebno voditi neku Kanban tabelu na GitHub-u ili drugom mestu?
    **Odgovor:** Ukoliko to Vama odgovara, možete, ali nije obavezno.
41. **Pitanje:** Da li nam možete dati primer apstrakcije issue-a. Da li je issue na nivou na primer CRUD operacije korisnika, repozitorijuma, i tako dalje ili da ih rasčlanimo na više manjih issue-a?
    **Odgovor:** Pored inicijalne postavke projekta, issue-i bi trebalo da budu vezani za funkcionalnosti (jedan issue - jedna funkcionalnosti). Npr. “registracija korisnika”, “prijava korisnika na sistem”, “pretraga repozitorijuma” itd., …
42. **Pitanje:** Da li je u redu da pokušamo autorizaciju uraditi uz pomoć OAutha preko githuba ili google naloga umesto da implementiramo naš sistem registracije?
    **Odgovor:** Trebalo bi imati svoj sistem registracije, ali korisniku se može dati mogućnost da se registruje putem Google ili nekog drugog naloga.
43. ~~**Pitanje:** Da li se kontrolne tačke ocenjuju?~~
    ~~**Odgovor:** Kontrolne tačke se ne ocenjuju direktno, ali indirektno svakako ulaze u ocenu. Naime, rad tokom kontrolnih tačaka nosi ukupno 10 bodova i~~

~~tiče se stavke *GitHub (aktivnost)* sledeće tabele. Da biste osvojili svih 10 bodova, neophodno je da svaka kontrolna tačka bude dobro isplanirana, tako što ćete se kao tim dogovoriti šta je potrebno uraditi tokom trajanja iste. Ovo dokumentujete u formi issue-a koje dodeljujete pojedinačnim članovima tima na izradu. Kontrolna tačka se smatra uspešno završenom, ukoliko su svi issue-a koji su planirani uspešno završeni i zatvoreni pre kraja roka definisanog *Milestone*-om. Ovo dovodi do toga da *GitHub* prikaže da je *Milestone* završen 100%.~~

44. ~~**Pitanje:** Šta ako isplaniramo da ne uradimo ništa tokom kontrolne tačke?~~

    ~~**Odgovor:** Kontrolna tačka koja ima 0 issue-a se smatra neuspešnom, što će dovesti do gubitka određenog broja bodova na samoj odbrani projektnog zadatka.~~

45. **Pitanje:** Koliko termina za odbranu projekta će biti?

    **Odgovor:** Ukupno 3. Prvi termin biće u junsko-julskom ispitnom roku (početkom avgusta prema novom radnom kalendaru). Naredni termin odbrane biće u avgustovkom roku (početkom oktobra). Poslednji termin odbrane u ovoj iteraciji kursa biće u septembarskom roku (sredinom oktobra). Nema skaliranja bodova te maksimalan broj bodova možete ostvariti u svakom ispitnom roku.

46. **Pitanje:** Da li će se smanjivati obim posla ukoliko tim broji manje od 3 člana?

    **Odgovor:** Ukoliko ste svojevoljno tražili da radite u timu od dva člana ili samostalno, tada nema smanjivanja obima posla. Ukoliko je pak kojim slučajem neko od kolega odustao od projektnog zadatka (ili odlučio da projekat radi u nekom od kasnijih rokova), u tom slučaju se obim posla skalira pod uslovom da tim na kraju broji manje od 3 člana i to na način opisan na dnu sledeće tabele. Timovi koji i nakon odustanka nekog od kolega i dalje broje bar 3 člana, treba da implementiraju sve stavke iz tabele za najvišu ocenu.

47. **Pitanje:** Budući da razmatramo da koristimo DockerHub API, da li je potrebno imati korisnika u našoj bazi ili možemo da se za to oslanjamo na DockerHub? Ukoliko imamo našeg korisnika u bazi, da li je potrebno omogućiti da on i bez povezivanja može da radi sa Docker slikama itagovima?

**Odgovor:** Neophodno je da Vaša aplikacija ima bazu u kojoj čuva korisnike registrovane na Vašu platformu. Korisnik može da radi sa repozitorijumima i tagovima samo ukoliko je registrovan i autentifikovan, pri čemu sam odlučuje o tome da li je repozitorijum javan ili privatan. Korisnik sve radi isključivo putem Vaše platforme i oni se perzistiraju u nekoj bazi koju aplikacija koristi, a Vi je administrirate. *DockerHub* se koristi isključivo kao remote repozitorijum. Drugim rečima, URL do *DockerHub*-a repozitorijuma koristite u docker CLI klijentu. *Push* (`docker push`) operacija će gađati ovaj repozitorijum, dok će Vaša aplikacija preko API-a dobiti informacije o tom repozitorijumu (npr. lista tagova, itd). O svim ostalim informacijama (kao što su korisnici, organizacije, itd.) brine Vaša aplikacija.

48. **Pitanje:** Da li bi bilo dozvoljeno da za hostovanje repozitorijuma koristimo kontejner čiji je image nečiji tuđi za Harbor sa DockerHub-a?
    **Odgovor:** Da.
49. **Pitanje**: Ukoliko se odlučimo da lokalno hostujemo container registry, da li je neophodno da bude podržana autentifikacija i autorizacija na njemu ?
    Odgovor: Neophodno je obezbediti autentifikaciju i autorizaciju te ograničiti pristup repozitorijumima.
50. ~~**Pitanje**: Da li svi članovi tima moraju biti prisutni na kontrolnoj tački?~~
    ~~Odgovor: Da.~~
51. **Pitanje**: Da li je potrebno pisati testove i za frontend?
    Odgovor: Ne.

**Odricanje od odgovornosti:** Tokom izrade projektnog zadatka, potrebno je upoznati se sa *DockerHub* platformom i Vašu aplikaciju implementirati u skladu sa istom. Ovaj dokument ne opisuje detaljno sve funkcionalne i nefunkcionalne zahteve, već sadrži smernice i pojašnjenja koja su nastala na osnovu pitanja studenata upućenih predmetnim asistentima kroz prethodne iteracije kursa. Asistenti tekuće iteracije kursa odriču se bilo kakve odgovornosti, ukoliko ovaj dokument sadrži propuste. Ukoliko imate bilo kakvih nejasnoća koja se tiču projektnog zadatka, ne bi trebalo donositi pretpostavke. Umesto toga, obratiti se predmetnom asistentu, odnosno profesoru, u cilju otklanjanja nejasnoća.